

Introduction to Python



Charles Forsyth
HPC/Linux System Administrator
High Performance Computing Center
University of California, Riverside
forsythc@ucr.edu

HPCC Website

Website Address: hpcc.ucr.edu



- Home ▲
- Introduction
- Activity report
- Slides
- News ▼
- Access & rates ▼
- Documents ▼
- Contacts ▼
- Events ▼
- Hardware ▼
- Software ▼
- Manuals ▼

High-Performance Computing Center @ UCR

Nav Related Websites Feedback Google Custom Search

High-Performance Computing Center (HPCC)

Summary: welcome to the website of the High-Performance Computing Center (HPCC) at UC Riverside. This site gives an overview of the HPC resources and services provided by our center.

Table of Contents

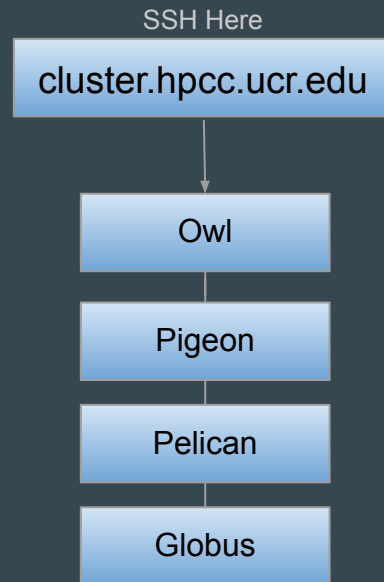
- Mission and Services
- HPC hardware
- Software
- Training
- Cloud and national HPC facilities

Mission and Services

The High-Performance Computing Center (HPCC) provides state-of-the-art research computing infrastructure and training accessible to all UCR researchers and affiliates at low cost. This includes access to the shared HPC resources and services summarized below. The main advantage of a shared research computing environment is access to a much larger HPC infrastructure (with thousands of CPUs/GPUs and many PBs of directly attached storage) than what smaller clusters of individual research groups could afford, while also providing a long-term sustainability plan and professional systems administrative support.

Accessing the Cluster - SSH

- SSH is the main protocol used to interface with the HPCC Cluster.
- Start by ssh'ing from your system to `cluster.hpcc.ucr.edu`
 - Windows
 - Putty or Mobaxterm
 - `ssh username@cluster.hpcc.ucr.edu`
 - Mac
 - Terminal
 - `ssh username@cluster.hpcc.ucr.edu`
 - Linux
 - Terminal
 - `ssh -XY username@cluster.hpcc.ucr.edu`
 - The “-XY” allows graphical output to be passed back to your system.
- `cluster.hpcc.ucr.edu` is a special DNS name that load balances incoming connections to one of the available login nodes.



Python introduction workshop files

- Git (preferred)
 - git clone https://github.com/ucr-hpcc/hpcc_python_intro.git
 - cd hpcc_python_intro/
- wget
 - wget https://github.com/ucr-hpcc/hpcc_python_intro/archive/master.zip
 - unzip master.zip
 - cd hpcc_python_intro-master/

Python: Overview

Python is a remarkably powerful programming language that is used in a wide variety of application domains. Python is often compared to Tcl, Perl, R or Ruby.

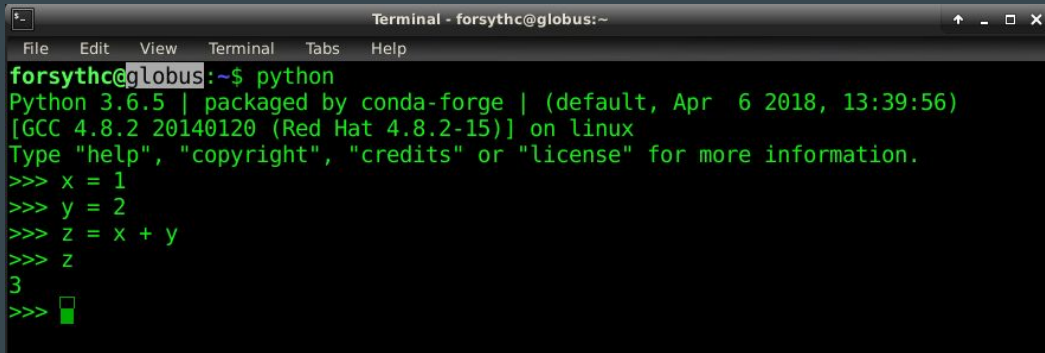
- High-level
- Interpreted
- Interactive
- Object-Oriented
- Can create GUI interfaces
- Extendable
- Easy to learn
- Easy to read

Python: Overview

- Python plays well with others
 - Python can integrate with R, COM, .NET, C and C++ just to name a few
- Python runs everywhere
 - Python is available for all major operating systems: Windows, Linux/Unix, OS/2, Mac
- Strong Community and Open Source
 - Many resources and code examples online
- Very powerful
 - Very rich standard library, which covers everything from asynchronous processing to zip files. The language itself is a flexible powerhouse that can handle practically any problem domain.
- Fast
 - Thanks to a highly optimized byte compiler and support libraries, Python code runs more than fast enough for most applications.

Python: Interpreter

- The python interpreter takes Python scripts and interprets them into machine code for you.
- Takes the high level language and translates it to something the processor can understand.
- Can allow work interactively with the interpreter to experiment and test.



```
Terminal - forsythc@globus:~  
File Edit View Terminal Tabs Help  
forsythc@globus:~$ python  
Python 3.6.5 | packaged by conda-forge | (default, Apr 6 2018, 13:39:56)  
[GCC 4.8.2 20140120 (Red Hat 4.8.2-15)] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> x = 1  
>>> y = 2  
>>> z = x + y  
>>> z  
3  
>>>
```

Python: Python Environments on the HPCC Cluster

- Two major python versions - python 2 and python 3 (python 3 is the future)
- Python Environment = Isolated Python space (python, modules and apps)
- Default = miniconda2/4.4.10
 - python = python3 (version 3.6.5)
 - python3 = python3 (version 3.6.5)
 - python2 = python2 (version 2.7.14)
- Create your own python environment:
 - `conda create -y -n py3.7 python=3.7.0`
 - `source activate py3.7`

Python: scripts

- Python scripts are just text files with python code in them.
 - File extension (.py) not necessary but good practice
- First line in the file defines the interpreter (python in this case)
 - `#!/usr/bin/env python`
- Permissions - At least the owner of the file requires execute permissions.
 - `chmod u+x my-python-script.py`

Python: Variable Types

- Variables are reserved memory locations to store values. This means that when you create a variable you reserve some space in memory. Depending on the data type the interpreter decides what can be stored in that reserved memory.
- Python has five standard data types:
 - Numbers
 - Strings
 - List
 - Tuples
 - Dictionary

Python: Variables - Numbers

number-type-variables.py

Python supports four different numerical types:

- `int` (integers)
- `long` (long integers)
- `float` (floating point real values)
- `complex` (complex numbers)

```
#!/usr/bin/env python

x = 1          # integer
y = 1.5        # float

x = 7          # integer
y = 2          # integer
z = x/y        # division
print(z)       # print the value of y
#3            # result , integer(python2)
#3.5          # result , float(python3)

x = 7.0        # float
y = 2.0        # float
z = x/y        # division
print(z)       # print the value of y
#3.5          # result , float
```

Python: Variables - Strings

string-type-variables.py

Strings are sets of characters
between quotation marks.

- Can be iterated over
- Can be referenced by index

```
#!/usr/bin/env python

mystr = 'Hello World!'

print(mystr)           # Prints complete string
print(mystr[0])        # Prints zeroth char of the string
print(mystr[2:5])      # Prints characters 3rd to 6th
print(mystr[2:])       # Prints characters 3rd to end
print(mystr * 2)       # Prints string two times
print(mystr + "TEST")  # Prints concatenated string
```

Python: Variables - Lists

`list-type-variables.py`

A list is a variable type that contains items separated by commas and enclosed within square brackets [].

- Can be iterated over
- Can be referenced by index
- Each entry in the list can be any variable type.
 - Even a list of lists will work

```
#!/usr/bin/env python

listone = [ 'abcd', 786 , 2.23, 'dog', 70.2 ]

listtwo = [123, 'dog']

print(listone)           # Prints complete list
print(listone[0])        # Prints zeroth element of the list
print(listone[1:3])      # Prints elements from 2nd to 4th
print(listone[2:])       # Prints elements from 3rd element
print(listtwo * 2)        # Prints list two times
print(listone + listtwo) # Prints concatenated lists

print(len(listtwo))      # Prints # of elements in listtwo
print(listtwo)           # Prints listtwo

listtwo[0] = 456         # Replaces zeroth element
listtwo.append('cat')    # Appends cat to listtwo

print(len(listtwo))      # Prints # of elements in listtwo
print(listtwo)           # Prints listtwo
```

Python: Variables - Tuples

[list-type-variables.py](#)

A tuple contains items separated by commas and enclosed within parentheses ().

They are very similar to lists.

The big difference is they are immutable, think **read-only**. The contents can not be changed.

```
#!/usr/bin/env python

tupleone = ( 'abcd', 786 , 2.23, 'dog', 70.2 )

tupletwo = (123, 'dog')

print(tupleone)           # Prints complete list
print(tupleone[0])        # Prints zeroth element of the tuple
print(tupleone[1:3])      # Prints elements from 2nd to 4th
print(tupleone[2:])       # Prints elements from 3rd on
print(tupletwo * 2)       # Prints tuple two times
print(tupleone + tupletwo) # Prints concatenated tuple

# tupleone[0] = 345        # This will FAIL, immutable
# tupleone.append('cat')   # This will FAIL, immutable
```

Python: Variables - Dictionary

dictionary-type-variables.py

A dictionary contains key-value pairs separated by commas and enclosed within curly brackets { }.

Can be iterated over, referenced by key, changeable and indexed

```
#!/usr/bin/env python

dictionary = {'name': 'John', 'code': 6734, 'dept': 'sales', 2: 'a number'}

print(dictionary['name'])      # Prints value for 'name' key
print(dictionary[2])          # Prints value for 2 key
print(dictionary)             # Prints complete dictionary
print(dictionary.keys())      # Prints all the keys
print(dictionary.values())    # Prints all the values

dictionary['name'] = 'Paul'    # Changes the value of name to Paul
dictionary['company'] = 'Big Company' # Adds new key-value pair
print(dictionary)             # Prints complete dictionary
```

Python: Flow Control

flow-control-sudo-code.txt

Like most other languages Python has the basic logic constructs.

- IF
 - IF
 - ELIF
 - ELSE
- WHILE
- FOR

```
##### Sudo Code #####

if condition:                                #Checks if condition is true
    action                                   #If true performs action
elif condition:                             #If false check this condition
    action                                   #If true performs action
else                                         #If no conditions are true
    action                                   #Perform this action

while condition:                            #while something is true
    action                                  #perform this action

for iterating_var in sequence:              #iterate over a sequence
    action on iterating_var                #perform action each iteration
```

Python: Lines and Indentation

`indentation-example.py`

Blocks of code are denoted by line indentation, which is rigidly enforced.

The number of spaces in the indentation is variable, but all statements within the block must be indented the same amount.

Four spaces per/indentation is best practice.

```
#!/usr/bin/env python

myvar = 42

if myvar == 42:
    print('Condition was True')
    mylist = ['red', 'green', 'blue']
    for var in mylist:
        print(var)
    print('done true list')
else:
    print('Condition was False')
    mylist = [1,2,3]
    for var in mylist:
        print(var)
    print('done false list')
print(myvar)
```

Python: Comparison Operators

Testing conditions in IFs, WHILEs and FORs requires a means to compare objects. This is done with operators, namely comparison operators.

Here is a chart of common comparison operators and what they do:

Operator	Description	Example
==	Checks if the value of two operands are equal or not, if yes then condition becomes true.	(a == a) is true
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	(a != b) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(5 > 2) is true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(1 < 2) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(2 >= 2) is true, (5 >= 2) is true
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(2 <= 2) is true, (1 <= 2) is true

Python: Comparison Operators

comparison-operators.py

Examples:

Comparison Operators in action

```
#!/usr/bin/env python

dictionary = 'yes'
list = 'yes'
threshold = 30

if dictionary == 'yes':
    print('Starting Dictionary Test')
    data = {'result-1':42,'result-2':100,'result-3':20}
    for result in data:
        if data[result] <= threshold:
            print(result, data[result])

if list == 'yes':
    print('Starting List Test')
    data = [2,4,3,6,9,100,42,20,35,68,39]
    for result in data:
        if result >= threshold:
            print(result)

print('Script Complete')
```

Python: Functions

functions.py

Blocks of code that you can define.

You can pass input to them and the result can optionally be returned.

Allows reuse of code

Syntax:

```
def function_name( parameters ):  
    code block  
    return [expression]
```

```
#!/usr/bin/env python  
  
def my_function():  
    print("Hello from a function")  
  
def my_greet_function(name):  
    print("Hello " + name + ", I am function")  
  
def my_pi(num):  
    num = num * 3.14  
    print(num)  
  
def my_multiply(num1,num2):  
    result = num1 * num2  
    return result  
  
my_function()  
my_greet_function('Chuck')  
for iteration in range(82, 101):  
    my_pi(iteration)  
print(my_multiply(566765, 8503432))
```

Python: Classes

classes.py

Most things in Python are objects with properties and methods.

A Class is like a "blueprint" for creating objects.

All classes have a function called `__init__()`

Objects can also contain methods. Methods in objects are functions that belongs to the object.

```
#!/usr/bin/env python

class Person:
    def __init__(self, name, id):
        self.name = name
        self.id = id

    def greet(self):
        print("Hello " + self.name)

    def report_id(self):
        print("Id =" + str(self.id))

p1 = Person("John", 36)
p2 = Person("Paul", 32)
p3 = Person("Kim", 33)
p1.greet()
p3.greet()
p2.report_id()
p3.report_id()
```

Python: Modules and Installs

- A module is a file(s) consisting of Python code.
 - A module can define functions classes, and variables.
 - You can define your own modules.
 - Python has a large number of modules available from the Python community.
 - Extends the language.
- You can install modules via conda, pip, or write them yourself.
 - `conda install <module name>`
 - `pip install <module name>`
 - placing `__init__.py` in the same directory as the python script your developing.
- To use a module you need to import that module into your code.
 - This is done with the “import” statement

Python: Modules - Import

module-import.py

To use a module once installed you need to import that module into your code.

- This is done with the “import” statement
- syntax:
 - import module
 - import module as name

```
#!/usr/bin/env python

import random           #Importing the random module

x = random.randint(1,10) #Generate random integer from 1 to 10
print(x)                 #Prints x
```

Python: Filesystem Operations

filesystem-operations.py

- You may wish to perform operations on a file on your filesystem.
 - Delete or Rename a file
 - Make a directory or find out your current working directory.
- These can be done by importing another module named OS.

```
#!/usr/bin/env python
```

```
import os                                #importing the os module

os.rename(current_file_name, new_file_name) #Renames a file
os.remove(file_name)                       #Deletes a file
os.mkdir('newdir')                         #Creates a directory
os.getcwd()                               #Returns your current working directory
os.chdir('newdir')                         #Changes your current working directory
os.mkdir('newdir1')                        #Creates a directory
os.mkdir('newdir2')                        #Creates a directory
os.mkdir('newdir3')                        #Creates a directory
os.rmdir('newdir2')                        #Deletes a directory
```

Python: External Commands

external-commands.py

Things such as top, sbatch, squeue or any other executable code outside of python. These can be done by importing another module named subprocess. The method is called call and it's part of the subprocess module

```
subprocess.call(['command to run', arguments'])
```

```
#!/usr/bin/env python
```

```
import subprocess
```

```
#importing the subprocess module
```

```
subprocess.call(['top'])
```

```
#calling external command top, type 'q' to exit
```

```
subprocess.call(['ls', '-latr'])
```

```
#calling external command ls -latr
```

```
subprocess.call(['./dictionary-type-variables.py'])
```

```
#calling other python script in same folder
```

Python: User input

user-input.py argument

Command line arguments

Interactive input

```
#!/usr/bin/env python

import sys

# Interactive Input
name = input('What is your name?\n')
print('Hi, %s.' % name)

#Command line arguments
if len(sys.argv) > 1:
    print(sys.argv[1])
```

Python: Read and Write Files

read-write-files.py

You can use the open function and what's called a file handle. To load a file into a file handle for reading, writing or both.

json is a module that lets you store data in json (JavaScript Object Notation) format.

Much like a dictionary.

There are many other ways to store data in files.

```
#!/usr/bin/env python

import json

data = {'r1':42,'r2':100,'r3':20}
data2 = {}

with open('data.txt', 'w') as file:
    file.write(json.dumps(data))

with open('data.txt', 'r') as file:
    data2= json.loads(file.read())

print(data2)
```

Python: Work with data

work-with-data.py

This example creates a data file if one doesn't already exist. A file with 1,234,567 results, random values between 1 and 314.

It reads in the data file, storing it in a dictionary. It then iterates over it storing the values in a list.

It then sums and averages the values.

```
#!/usr/bin/env python

import os
import random
import json
if not os.path.isfile('data.txt'):
    data = {}
    for i in range(1234567):
        data['r'+ str(i)] = random.randint(1,314)
    with open('data.txt', 'w') as file:
        file.write(json.dumps(data))

with open('data.txt', 'r') as file:
    data = json.loads(file.read())

l = []
for result in data:
    l.append(data[result])
average = sum(l) / len(l)
print('The sum is ', round(sum(l), 2))
print('The average is ', round(average, 2))
```

Python: Python in HPC Environments

`python-slurm.py`

- Very Popular
- Batch submissions can be made in python
 - `sbatch python-slurm.py`
- GPU and Machine Learning
 - [pytorch](#)
 - tensorflow
 - tflearn
 - [scikit-learn](#)
- General HPC
 - numpy
 - scipy
 - Multiprocessing
 - Mpi4py

Thank You

Helpful Resources:

- Command help
 - `dir(class)`
 - `help(class.function)`
- Web Resources
 - [Python on W3 Schools](#)
 - [Stack Overflow](#)
 - [Google for Education](#)
 - Google “[python <my question>](#)”